ML Experiment Triage Environment

A Reinforcement Learning Environment for ML Experiment Analysis

Charan Sai Ponnada
Scaler School of Technology

April 9, 2026

Abstract

This document provides comprehensive technical documentation for the ML Experiment Triage Environment, an OpenEnv-compliant reinforcement learning environment designed for automating the analysis and triage of machine learning experiment results. The system enables AI agents to identify best-performing experiments, detect overfitting, and suggest optimal hyperparameter configurations through a structured action-space interface. This environment was developed for the Meta PyTorch Hackathon and implements state-of-the-art practices in RL environment design.

Contents

1 Introduction

1.1 Background and Motivation

Machine learning development involves running numerous experiments with varying hyperparameters, architectures, and training strategies. Manually reviewing each experiment's results to identify the best configurations is time-consuming and error-prone. The ML Experiment Triage Environment addresses this challenge by simulating an RL environment where an AI agent learns to efficiently triage experiment results.

1.2 Problem Statement

Given a set of ML experiment results, the agent must:

- **Identify the best experiment** among multiple runs
- **Detect overfitting** patterns in training metrics
- **Suggest next configurations** based on observed patterns
- **Compare experiments** and analyze performance trade-offs
- **Debug failed runs** and diagnose root causes

1.3 Contributions

The main contributions of this work are:

1. An OpenEnv-compliant RL environment with structured action and observation spaces
2. Five distinct tasks covering various ML experiment analysis scenarios
3. A Gradio-based web interface for human-in-the-loop evaluation
4. Docker containerization for easy deployment and evaluation
5. Integration with LiteLLM proxy for standardized LLM inference

2 System Architecture

2.1 Overview

The system consists of three main components:

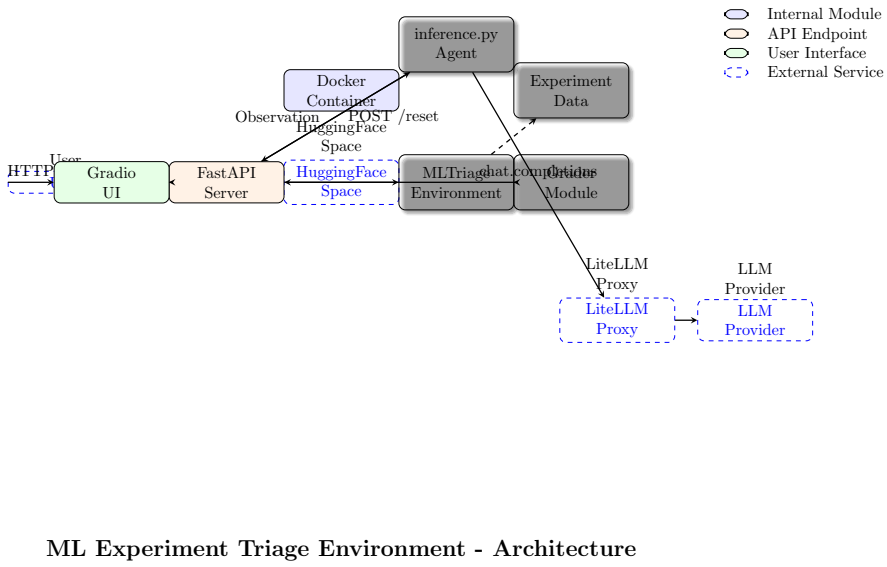

Figure 1: System Architecture Diagram

2.2 Component Description

2.2.1 Environment Server

The environment server (`server/`) implements the OpenEnv interface:

- `ml_triage_environment.py`: Core environment implementation
- `app.py`: FastAPI application with OpenEnv integration
- Handles reset, step, and state operations

2.2.2 Web Interface

The web interface (`app/`) provides a Gradio-based UI:

- `main.py`: FastAPI server with static file serving
- `env.py`: Custom environment with LLM integration
- `templates/index.html`: Interactive Gradio interface
- Real-time experiment visualization and action feedback

2.2.3 Inference Pipeline

The inference pipeline (`inference.py`) runs autonomous evaluation:

- Connects to environment via REST API
- Uses LLM for action decision-making
- Outputs structured logs for evaluation

3 Environment Specification

3.1 Observation Space

The observation space is a structured dictionary containing:

Table 1: Observation Space Fields

Field	Description
`experiments`	List of ExperimentRecord objects
`current_step`	Current step number (0-indexed)
`max_steps`	Maximum allowed steps for the task
`task_description`	Natural language task description
`feedback`	Feedback from the previous action
`serialized_state`	Additional state information

3.2 Action Space

The action space consists of four discrete actions:

1. `investigate(exp_id)`: Examine an experiment's details
2. `discard(exp_id)`: Mark an experiment as not worth keeping
3. `suggest(suggestion)`: Propose next hyperparameter configuration
4. `summarize(summary)`: Provide final answer and end episode
5. `compare(comparison)`: Compare two experiments (Tasks 4)
6. `diagnose(diagnosis)`: Diagnose a failed run (Task 5)

3.3 Experiment Record Schema

```
@dataclass
class ExperimentRecord:
    exp_id: str            # Unique experiment identifier
    model_name: str         # Model architecture name
    learning_rate: float    # Learning rate
    epochs: int            # Number of training epochs
    train_acc: float       # Training accuracy
    val_acc: float        # Validation accuracy
    train_loss: float      # Training loss
    val_loss: float       # Validation loss
    notes: str            # Additional notes
    status: str           # Status: pending, investigated,
       discarded
```

Listing 1: ExperimentRecord Model

4 Task Specifications

4.1 Task 1: Find Best Experiment

Difficulty: Easy

Objective: Identify the single best performing experiment from a set of 8 runs.

Grading Criteria:

- Correctly identify exp_004 as the best experiment: 0.9999
- Investigate exp_004 without summarizing: 0.5
- No correct identification: 0.0001

Action Pattern: The agent should investigate multiple experiments and use `summarize` with the correct exp_id.

4.2 Task 2: Identify Overfitting

Difficulty: Medium

Objective: Detect and discard all 3 overfitting experiments.

Overfitting Definition: An experiment is overfitting if:

$$\text{train_acc} > 0.97 \quad \text{and} \quad \text{val_acc} < 0.75 \tag{1}$$

Grading Formula:

$$\text{score} = \frac{\text{correct_discards}}{3} - 0.1 \times \text{wrong_discards} \tag{2}$$

Target Experiments: exp_002, exp_006, exp_009

4.3 Task 3: Suggest Next Experiment

Difficulty: Hard

Objective: Based on incomplete experiment results, suggest the next hyperparameter configuration.

Grading Criteria:

- All 3 fields correct (lr=0.001, epochs=50, model=resnet50): 0.9999
- 2 fields correct: 0.6
- 1 field correct: 0.3
- No fields correct: 0.0001

4.4 Task 4: Compare Experiments

Difficulty: Medium
Objective: Compare two experiments and analyze performance trade-offs.
Scoring Keywords:

- exp_004 mentioned: +0.4
- validation metrics analyzed: +0.2
- generalization discussed: +0.2
- trade-off analysis: +0.2

4.5 Task 5: Debug Failed Run

Difficulty: Hard
Objective: Diagnose the root cause of a failed experiment run.
Known Failure Modes:

- exp_005: Learning rate issues (high)
- exp_008: Memory/OOM issues
- exp_003: Training plateau issues

5 Reward System

5.1 Reward Structure

The environment uses dense rewards for step-by-step feedback:

Table 2: Reward Values by Action

Action	**Reward**
Successful investigate	+0.1
Correct discard (overfitting)	+0.05
Incorrect discard	-0.1
Invalid action	-0.05
Episode completion	0.0 to 1.0

5.2 Final Score Calculation

The final score is clamped to ensure strict bounds:

$$\text{final_score} = \max(10^{-9}, \min(1 - 10^{-9}, \text{sum}(\text{rewards}))) \tag{3}$$

6 Implementation Details

6.1 Server Implementation

```
def create_env():
    return MLTriageEnvironment()

app = create_app(
    create_env,
    MLTriageAction,
    MLTriageObservation,
    env_name="ml-experiment-triage"
)
```

Listing 2: Environment Factory

6.2 Grader Implementation

```
def grade_task_1(experiments, episode_history, summary):
    investigated_exp_ids = set()
    for step in episode_history:
        if step["action"]["action_type"] == "investigate":
            investigated_exp_ids.add(step["action"]["exp_id"])

    if summary and "exp_004" in summary:
        return _clamp_strict(0.9999)

    if "exp_004" in investigated_exp_ids:
        return 0.5

    return _clamp_strict(0.0001)
```

Listing 3: Task Grading Function

6.3 Inference Pipeline

```
def get_action(obs_text: str, history: list) -> dict:
    messages = [{"role": "system", "content": SYSTEM_PROMPT}]
    for h in history[-6:]:
        messages.append({"role": "user", "content": h["obs"]})
        messages.append({"role": "assistant", "content": h["
           action"]})
    messages.append({"role": "user", "content": obs_text})

    resp = client.chat.completions.create(
        model=MODEL_NAME,
        messages=messages,
        max_tokens=MAX_TOKENS,
        temperature=TEMPERATURE,
    )
```

```
    return json.loads(resp.choices[0].message.content)
```

Listing 4: LLM Action Generation

7 Deployment

7.1 Docker Container

The application is containerized using Docker:

```
FROM python:3.11-slim

WORKDIR /app
RUN pip install uv
RUN uv pip install --system fastapi openai openenv-core \
    pydantic pyyaml uvicorn jinja2 python-multipart requests

COPY server/ ./server/
COPY app/ ./app/
COPY models.py ./
COPY inference.py ./

EXPOSE 7860
CMD ["python", "-c", "from app.main import app; \
    import uvicorn; uvicorn.run(app, host='0.0.0.0', port=7860)"]
```

Listing 5: Dockerfile

7.2 HuggingFace Space

The environment is deployed as a HuggingFace Space:

- SDK: Docker
- Port: 7860
- Hardware: CPU (default)
- Repository: https://huggingface.co/spaces/Charansaiponnada/ml-experiment-triage-en

8 API Reference

8.1 REST Endpoints

8.1.1 POST /reset

Reset the environment for a new episode.

```
POST /reset
Content-Type: application/json

```

```
{"task_id": 1}
```

Listing 6: Reset Request

Response:

```
{
    "observation": {...},
    "reward": null,
    "done": false
}
```

Listing 7: Reset Response

8.1.2 POST /step

Execute an action in the environment.

```
POST /step
Content-Type: application/json

{"action": {"action_type": "investigate", "exp_id": "exp_001"}}
```

Listing 8: Step Request

8.1.3 GET /health

Health check endpoint.

```
{"status": "ok"}
```

Listing 9: Health Response

9 Evaluation Metrics

9.1 Score Calculation

Each task returns a score in the range $(0, 1)$:

- Scores of exactly 0.0 or 1.0 are rejected
- Epsilon padding (10^{-9}) ensures strict bounds
- Final submission score is the average across all tasks

9.2 Success Criteria

A submission is considered successful if:

1. Docker container builds successfully
2. inference.py executes without errors
3. Output parsing validates correctly

4. All task scores are in $(0, 1)$
5. LLM calls go through the provided LiteLLM proxy

10 Future Work

10.1 Potential Improvements

- **Multi-agent support**: Extend to allow multiple agents to collaborate
- **Custom datasets**: Allow users to upload their own experiment data
- **Advanced metrics**: Add more sophisticated evaluation metrics
- **Transfer learning**: Pre-train agents on synthetic data
- **Human feedback integration**: Allow human correction during episodes

11 Conclusion

The ML Experiment Triage Environment provides a robust framework for training and evaluating AI agents in the task of ML experiment analysis. Through its structured action space, diverse task set, and comprehensive evaluation system, it enables researchers to develop and benchmark intelligent agents for automating the tedious process of experiment triage.

The environment's compliance with the OpenEnv standard ensures compatibility with the broader ecosystem of RL environments, while its Docker and HuggingFace deployment options make it accessible for both local development and cloud-based evaluation.

Appendices

11.1 Appendix A: File Structure

```
ml-experiment-triage-env/
 app/
    main.py           # FastAPI web server
    env.py            # Custom environment with LLM
    models.py         # Pydantic models
    tasks.py          # Task definitions and graders
    data.py           # Experiment data generation
    templates/
        index.html    # Web interface
 server/
    app.py            # OpenEnv integration
    ml_triage_environment.py  # Core environment
 inference.py          # Autonomous evaluation script
 models.py             # Shared Pydantic models
 Dockerfile            # Container definition
 openenv.yaml         # Environment specification
 README.md            # Project documentation
```

11.2 Appendix B: Environment Variables

Table 3: Environment Variables

Variable	Default	Description
API_BASE_URL	router.huggingface.co/v1	LLM endpoint
API_KEY	-	Authentication key
HF_TOKEN	-	HuggingFace token
MODEL_NAME	Qwen/Qwen2.5-72B-Instruct	LLM model
ENV_BASE_URL	http://localhost:7860	Environment URL

11.3 Appendix C: Acknowledgments

This project was developed for the Meta PyTorch Hackathon x Scaler School of Technology, Round 1. We acknowledge the OpenEnv team for providing the environment framework and HuggingFace for hosting infrastructure.

References

1. OpenEnv Core Documentation. https://github.com/openenv/core
2. OpenAI API Reference. https://platform.openai.com/docs/api-reference
3. LiteLLM Documentation. https://docs.litellm.ai/
4. HuggingFace Spaces. https://huggingface.co/docs/hub/spaces
5. Gradio Documentation. https://gradio.app/docs/